

Reconstitution of the Heart Beat Engine: Generational Evolution, Packer Archival Schema, and Cross-Platform Localization Cipher

The preservation and localization of legacy Japanese Role-Playing Games (JRPGs) frequently require the reconstruction of proprietary game engines that were abandoned after their original development studios closed. This documentation provides a technical blueprint for the recovery and modification of the Heart Beat Engine (HBE), the specialized 32-bit software framework used for *Dragon Quest VII* (released in North America as *Dragon Warrior VII*) and the PlayStation remake of *Dragon Quest IV*. Designed as a unified, portable engine, this framework standardizes resource management, script execution, text decompression, and sound rendering across late-lifecycle PlayStation projects.

This report outlines the steps required to execute a "Frankenstein" localization project [User Query]. The process involves grafting the asset archive of *Dragon Quest IV* (HBD1PS1D.Q41) onto the executable runtime environment of the North American *Dragon Warrior VII* (SLUS-01206 or SLUS-01346). This analysis details the file format specifications, data schemas, and text-decoding algorithms needed to reconstruct the asset pipeline and complete the translation.

Archeology of the Dragon Quest Core Schema: From 8-Bit Cartridges to 32-Bit CD-ROM Architectures

Developing a unified schema for the *Dragon Quest* franchise requires tracing how its underlying data structures evolved across three console generations [User Query]. This evolutionary history shows a progression from highly constrained 8-bit assembly structures to modular, object-oriented 32-bit virtual databases.

```
+-----+
|                                     |
|               DATA EVOLUTIONARY TIMELINE                |
|                                     |
|      ----->          |                                     |
| - Highly optimized ASM           | - Larger ROM cartridges   |
|                                   |                             |
| - Bit-packed save slots         | - Hiragana to Kanji transition |
|                                   |                             |
| - Hardcoded variables           | - Structural stats        |
(Resilience)    |                                     |
|               |                                     |
|               |                                     |
```


Engine Feature	8-Bit NES Generation	16-Bit SNES Generation	32-Bit PSX Heart Beat Engine Schema
Storage Medium	ROM Cartridge (Discrete Banks)	ROM Cartridge (Extended Mappers)	CD-ROM (Unified Virtual Archive)
Character Sets	Hiragana Only (Single-Byte)	Hiragana, Katakana, and Kanji	Multibyte Shift-JIS & Tokenized Bitstreams
Save State Logic	Spell of Restoration / Battery SRAM	Battery-Backed Multi-Slot SRAM	Card-Based Binary Serialization
Data Organization	Hardcoded Assembly Offsets	Structured Tables and Pointers	Master-to-Sub-Block File Trees
Stat Calculations	Base Defense Derived from Agility	Dedicated "Resilience" Attribute	Multi-Class Dynamic Stat Matrix
<i>Event Scripting</i>	Inline Assembly Branches	Conditional Jump Tables	Tokenized Sub-Block Event Codes

Architectural Packer Specification for the Heart Beat Disc Format

The primary database for the Heart Beat Engine is stored in a single archive on the CD-ROM. This archive is named HBD1PS1D.Q41 for *Dragon Quest IV* and HBD1PS1D.Q71 for *Dragon Quest VII*. To maximize read performance on the PlayStation's double-speed CD-ROM drive, the archive uses a strict 2048-byte sector alignment. Every resource file inside the archive is aligned to a physical sector boundary, meaning any unused space at the end of a sector is padded with null bytes (0x00).

The first 2048 bytes (Sector 0) of the archive function as a generic volume descriptor. This sector is identical in both games, except for the volume string located at offset 0x740, which reads either hdb1ps1d.q41 or hdb1ps1d.q71. To calculate the absolute offset of any asset on the disc, developers can use the following equation:

$$O_{\{\text{file}\}} = S_{\{\text{LBA}\}} \times 2048$$

where $S_{\{\text{LBA}\}}$ represents the logical block address of the target sector. Inside the archive, assets are grouped into Master Blocks, which contain one or more Sub-Blocks. Every Master Block starts with a 16-byte header containing little-endian metadata.

Offset	Size (Bytes)	Format	Field Name	Description / Verification Rule
0x00	4	UInt32	sub_block_count	Number of child sub-blocks inside this Master Block
0x04 (start_span) (end_span)	4	UInt32	sector_span	Total size of the block, measured in 2048-byte sectors
0x08 (start_span) (end_span)	4	UInt32	data_length	Total size of all sub-blocks, excluding sector padding
0x0C (start_span) (end_span)	4	UInt32	null_pad	Reserved for future expansion (always set to

Offset	Size (Bytes)	Format	Field Name	Description / Verification Rule
				0x00000000)

Directly following the Master Block header is an array of 16-byte Sub-Block headers. For a Master Block containing N sub-blocks, the engine reads N consecutive 16-byte headers starting at offset 0x10.

Offset	Size (Bytes)	Format	Field Name	Description / Verification Rule
0x00	4	UInt32	compressed_size	Size of the sub-block payload as stored in the archive
0x04 (start_span)[span_113](end_span)	4	UInt32	decompressed_size	Size of the payload after decompression in Kernel RAM
0x08 (start_span)[span_114](end_span)	4	UInt32	asset_pointer	Internal ID or runtime pointer used by the script engine
0x0C (start_span)[span_115](end_span)	2	UInt16	compression_flag	Compression status flag (0 = Raw, 1280 = LZS Compressed)
0x0E	2	UInt16	sub_block_type	Type identifier that determines how the data is parsed

Approximately 75.5% of the sub-blocks in the archive are uncompressed (flag 0), while 24.5% are compressed using a proprietary Lempel-Ziv-Storer-Szymanski (LZS) algorithm (flag 1280). The efficiency of the LZS compression can be measured using the following compression ratio formula:

$$R_{\text{compression}} = \left(1 - \frac{S_{\text{compressed}}}{S_{\text{decompressed}}[span_21](start_span)[span_21](end_span)[span_37](start_span)[span_37](end_span)}} \right) \times 100\%$$

When the engine reads a sub-block marked with flag 1280, it allocates a temporary decompression buffer in memory of size decompressed_size, decodes the LZS stream, and then routes the raw data to the appropriate system manager based on its sub_block_type.

The Huffman Tree Cipher and Dialogue Control Engine

Text assets in the Heart Beat Engine are stored in Type 39 sub-blocks. Rather than using plain text strings, the engine compresses all dialogue into bit-packed streams using a custom Huffman coding algorithm. To read or replace dialogue, developers must parse a 24-byte Text Block Header that defines the boundaries of both the text stream and its decryption tree.

Offset	Size (Bytes)	Format	Field Name	Description / Verification Rule
0x00	4	UInt32	eof_pointer	Offset "a" pointing

Offset	Size (Bytes)	Format	Field Name	Description / Verification Rule
				near the end of the text block
0x04 (start_span) (end_span)	4	UInt32	scene_id	Unique identifier for the scene (e.g., the intro is 0x0000006C)
0x08 (start_span) (end_span)	4	UInt32	huffman_start	Offset "c" marking the start of the compressed bitstream
0x0C	4	UInt32	tree_start	Offset "d" marking the start of the Huffman Tree
0x10	4	UInt32	huffman_end	Offset "e" marking the end of the compressed bitstream
0x14 (start_span) (end_span)	4	UInt32	reserved	Reserved for future expansion (always set to 0x00000000)

Huffman Tree Parsing Mechanics

The Huffman tree is stored as a binary tree composed of nested node pairs. The total node count of the active tree is defined by a 2-byte integer located at the start of the tree structure, indicated by the E3 prefix.

Let T define the root node of the Huffman tree, and

b_i represent individual bits read sequentially from the bitstream starting at offset "c". The decoding routine navigates the tree by evaluating each bit :

- If $b_i = 0$, the pointer branches to the left child node.
- If $b_i = 1$, the pointer branches to the right child node.

This traversal continues until the decoder reaches a leaf node, which resolves to an 8-bit or 16-bit character code. The decoding loop can be mathematically expressed as a recursive function finding leaf L :

$$L = \text{Branch}(N_{\text{current}}, b_i) \quad \text{where} \quad N_{\text{current}} = \begin{cases} T & \text{if } i = 0 \\ \text{Branch}(N_{\text{current}}, b_{i-1}) & \text{if } N_{\text{current}} \notin \text{Leaves} \end{cases}$$

If the decrypted leaf yields a control character, the engine branches to a script interpreter instead of outputting a character to the VRAM tilemap generator.

Script Command Execution and Conditional

Evaluation Subsystems

The Dialogue Control Engine uses a custom formatting and control language to handle line breaks, text wrapping, and variable substitution. These control codes regulate text wrapping, page breaks, sound triggers (such as voice pitch modulation), and dynamic variable substitution.

All native Japanese dialogues require control blocks to manage line layout and text flow. The following table catalogs the core control codes found in the system.

Control Code (Hex)	Operational Meaning	System Behavior and Execution Details
0000	End of Dialogue	Terminates the current text stream and closes the message window.
7F02	New Line & Tab	Forces a carriage return and indents the cursor for the next text line.
7F04	Name Decorator	Toggles character name banners or switches text to italic/accent styling.
7F0A	End of Line Signifier	Confirms the end of a line, prompting the engine to wait for player input.
7F0B	Alt End of Line	Secondary line-end marker, used for automatic scrolling sequences.
7F0C	Clear Screen Cluster	Clears the dialogue box; typically written in consecutive blocks of six.
7F15	Gold Acquisition	Dynamic string interpolation that displays the current party gold balance.

Dialogue Formatting Variables

For scene scripting, Type 39 blocks execute script commands directly embedded in the text stream. The most prominent command is C021A0, which serves a dual role :

- C021A0 <offset> <dialogId>: Directs the game engine to jump to a specific dialogue index located at a designated file offset.
- C021A0 <FFF0> <key>: Evaluates a quest state or item check, dynamically routing the dialogue based on the player's inventory or progress.

To facilitate translation into Western languages with complex grammatical rules, the system supports a conditional formatting syntax. This is particularly critical when porting scripts designed for the mobile versions of the game, which use a tokenized logic structure :

- **Character Exclusion/Inclusion (%A, %B):** %A{ID} executes the subsequent block if the active party leader is *not* the character matching the ID. %B{ID} acts as an "else" clause, executing the text if the active party leader *is* that character.
- **Conditional Boundaries (%X, %Z):** %X marks the start of a conditional text string, and

%Z terminates it.

- **Pluralization Logic (%H):** %H{ID} evaluates an integer variable (e.g., gold coins or item quantities). If the value is greater than one, the engine dynamically appends the suffix wrapped in the %X and %Z bounds.

This variable engine structure is illustrated below:

```
%A120%XStill, if she were a boy-%Z%B120%XStill, if you were a boy-%Z  
Torneko receives %a00530 gold coin%H530%X%Ys%Z.
```

In this code sequence, the system dynamically checks the active character ID. If the character is not Alena (ID 120), the engine displays "Still, if she were a boy—". If the character is Alena, it prints "Still, if you were a boy—". Similarly, the pluralizer %H530 appends "s" to "gold coin" if the received quantity exceeds one.

The Sound Driver and Waveform Streaming Architecture

The sound system in the Heart Beat Engine does not use the standard Sony PlayStation sequence (SEQ) or instrument bank (VAB) driver formats. Instead, it uses a proprietary driver designed to share instrument wave banks across multiple sequence tracks, saving system memory.

Each sound block contains a 60-byte master header, which is followed by up to four digital wave bank descriptors (VB / PSX-ADPCM format) and a custom sequence block known as SEQq.

Offset	Size (Bytes)	Format	Description
0x00	4	UInt32	Sequence Data (SEQq) payload size (set to 0 if the block is instrument-only)
0x04	2	UInt16	Track Sequence ID (msq_id)
0x06	1	UInt8	Active Instrument Wave Bank count (typically up to 4)
0x07	1	UInt8	RAM Load position index (0 for constant BGM address space)
0x08	4	UInt32	Reserved / Pad
0x0C	4	UInt32	Wave Bank #1: Size of the PSX-ADPCM waveform data
0x10	4	UInt32	Wave Bank #1: Size of the regional mapping array
0x14	2	UInt16	Wave Bank #1: Device slot index
0x16	2	UInt16	Wave Bank #1: Parameter config flags

Offset	Size (Bytes)	Format	Description
0x18	12	Array	Wave Bank #2: Configuration block (identical to Bank #1)
0x24	12	Array	Wave Bank #3: Configuration block (identical to Bank #1)
0x30	12	Array	Wave Bank #4: Configuration block (identical to Bank #1)

At runtime, the sound loader reads these assets incrementally from the CD. The sequence data is loaded into the BGM driver buffer spanning RAM addresses 0x800D25C0 to 0x800D80EF. The sequence executes starting at address 0x80138000, while the waveform data is streamed directly to SpuRAM starting at 0x80180000.

The SEQq Music Format Specification

The sequence payloads start with the magic identifier qQES. While similar to standard MIDI-based PSX SEQ structures, SE[span_185](start_span)[span_185](end_span)Qq features a modified header and a distinct termination command.

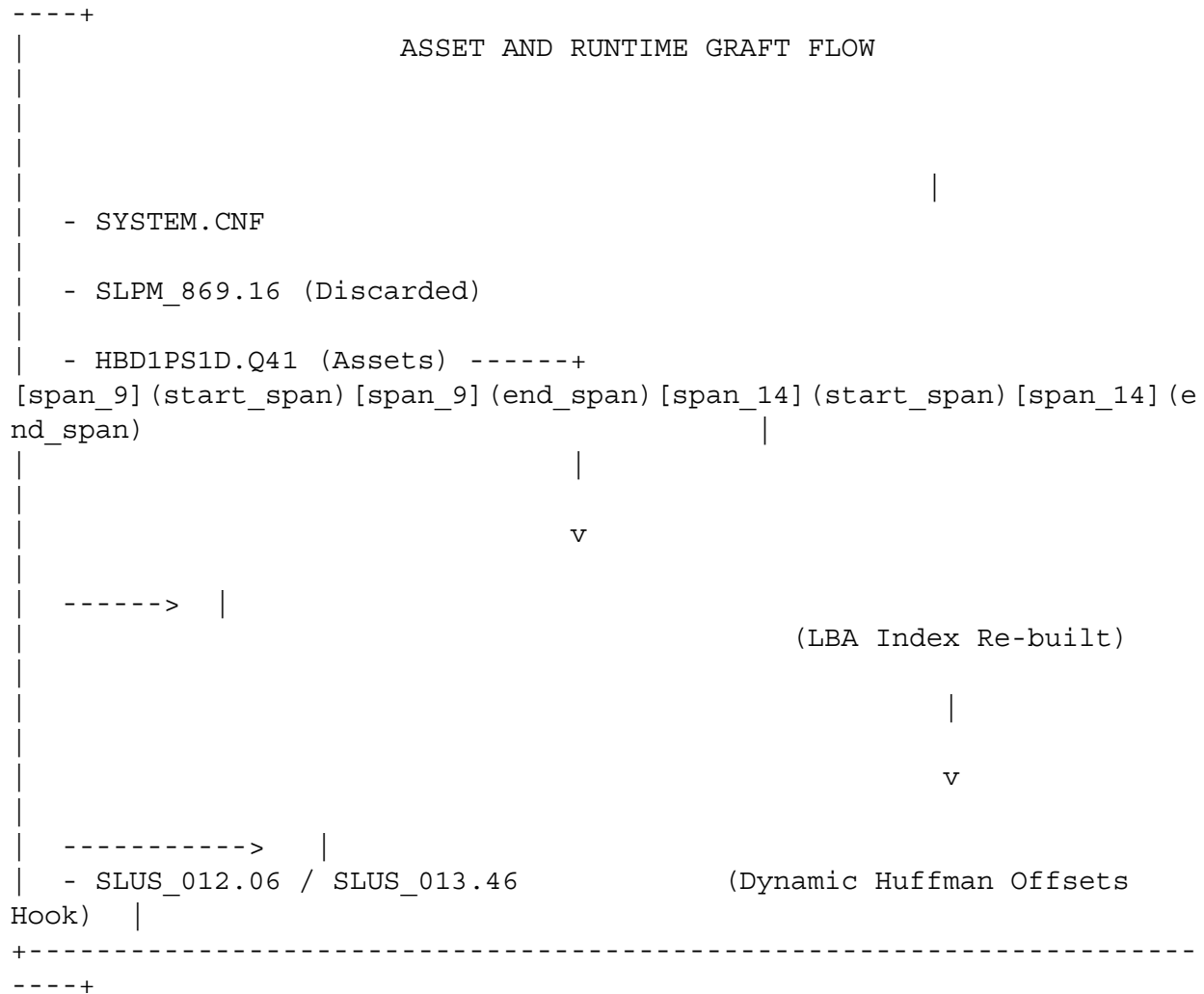
Field Name	Size (Bytes)	Expected Value / Description
Signature	4	qQES (ASCII)
Version	2	0x0000
Track Number	2	Sequence mapping ID (e.g., 0x1001)
TPQN	2	Ticks Per Quarter Note resolution
Tempo	3	Microseconds per quarter note
Numerator	1	Time signature numerator
Denominator	1	Time signature denominator (represented as 2^n)
Channels	1	Number of active sound channels used by the track

Unlike standard MIDI or PSX sequences, which rely on end-of-track commands prefixed with delta-time arguments, a SEQq stream terminates abruptly with the absolute byte sequence FF 2F 00 without any delta-time prefix. Standard parser tools will crash or hang on this sequence unless the extraction tool strips this marker and appends a standard null delta-time byte during conversion.

System-Level Guidelines for the Frankenstein Localization Graft

Executing a localization graft using the English *Dragon Warrior VII* executable (SLUS_012.06 or SLUS_013.46) to load assets from the modified *Dragon Quest IV* archive (HBD1PS1D.Q41) requires several system-level adjustments to ensure compatibility.

+-----



1. Archive Filename and Volume Header Synchronization

The target executable expects to open an archive named HBD1PS1D.Q71. The developer must rename the modified HBD1PS1D.Q41 archive to match this expected name before building the disc image.

Additionally, Sector 0 of the archive must be edited. The volume identifier string at offset 0x740 must be changed from hdb1ps1d.q41 to hdb1ps1d.q71 so that the executable's CD-ROM driver can verify and open the file.

2. Executable Logical Block Address (LBA) Reconstruction

The files within the modified archive will be located at different sector offsets compared to the original *Dragon Warrior VII* assets.

The developer must locate the hardcoded LBA index table inside the SLUS executable. The patching utility must scan the new archive, catalog the starting sector and total sector span of every Master Block, and overwrite the values in the executable's LBA table with the new sector map. This ensures that when the engine attempts to load an asset (such as Map 45), the

CD-ROM driver reads from the correct sector of the grafted archive.

3. Dynamic Huffman Tree Decoding Patch

The *Dragon Warrior VII* executable is compiled with a text-decoding routine designed to navigate its own hardcoded English Huffman tree. If the *Dragon Quest IV* Japanese assets are loaded directly, the text decoder will output corrupt data because it will attempt to decode the Japanese characters using the English tree.

To resolve this, the developer must modify the text decoding routine inside the executable. Instead of using the executable's hardcoded English tree, the text decoder must be updated to dynamically read the Huffman tree offset directly from offset 0x0C of each text block's header. This change ensures that the engine decodes each dialogue segment using the specific Huffman tree packed alongside that dialogue block.

4. Memory Allocations for English Dialogue Expansion

English translations typically require more characters—and therefore more bytes—than the original Japanese text. When injecting the English translation, the decompressed size of the sub-block (U_L) will likely exceed its original size.

If a translated sub-block's decompressed size is larger than its original value, the developer must update both the Master Block header and the corresponding Sub-Block header with the new size values. This prevents memory buffer overflows and crashes when the engine allocates RAM for the decompressed text.

- **Sub-Block Header Updates:** Update the compressed data length at offset 0x00 and the uncompressed data length at offset 0x04.
- **Master Block Header Updates:** Re-calculate the total raw data length, update offset 0x08, and pad the end of the Master Block with zeros (0x00) to ensure it remains aligned to a 2048-byte sector boundary.
- **Logical Block Address Updates:** If a Master Block's sector count increases, update the sector span at offset 0x04 of the Master Block header and adjust the starting sector offsets of all subsequent blocks in the executable's LBA index.

5. Audio Mapping Adjustments

The sound sequencer files (SEQq) and instrument wave banks are compatible between both games because they share the same underlying driver architecture.

However, because the instruments and sound effects are stored at different indexes in the two games, the developer must ensure that the instrument bank mapping table in the executable matches the layout of the grafted archive. If instrument mapping issues occur, the developer can patch the instrument bank sizes at offsets 0x0C, 0x18, 0x24, and 0x30 in the sound format header to force the engine to load the correct wave banks.

Reconstitution Parameter Specifications

By following this implementation strategy, developers can bypass the need to reverse-engineer the entire game engine. Utilizing the *Dragon Warrior VII* English executable as a base engine provides a stable foundation for loading and running *Dragon Quest IV* assets.

The table below outlines the core configuration parameters that must be set in the patching utility to ensure the grafted archive aligns correctly with the target executable.

Parameter Name	Target Configuration Location	Functional Mapping Value	Purpose of Configuration
Archive Target File	CD-ROM root directory	HBD1PS1D.Q71	Overrides game-disc file detection logic.
System Identifier	Sector 0, Offset 0x740	hdb1ps1d.q71 (ASCII value)	Validates the archive name with the engine's filesystem driver.
Master LBA Index	SLUS-01206 / SLUS-01346 EXE RAM	Re-calculated Sector Positions	Directs the CD-ROM driver to the correct assets.
Huffman Root Address	Text Block Header, Offset 0x0C	Offset variable "d"	Enables dynamic, per-scene text decompression.
Buffer Memory Cap	Master Header, Offset 0x08	Calculated Byte Count (U_L or C_L)	Prevents memory allocation crashes during decompression.
Driver SPU Target	SPU Audio Driver, SpuRAM Offset	0x80180000 (Hardware Address)	Loads instrument waveform assets to the SpuRAM buffer.

Works cited

1. does anyone know what happened to the planned DQ IV PS1 release? Did this eventually turn into the DS release? : r/dragonquest - Reddit, https://www.reddit.com/r/dragonquest/comments/vp7xdv/does_anyone_know_what_happened_to_the_planned_dq/ 2. Dragon Quest VII - Wikipedia, https://en.wikipedia.org/wiki/Dragon_Quest_VII 3. Dragon „Hackst“ IV | Markus Projects, <http://markus-projects.net/dragon-hackst-iv/> 4. PlayStation (PS1) Dragon Quest VII and IV Music Format (SEQQ) - loveemu labo, https://loveemu.hatenablog.com/entry/20140103/PlayStation_Dragon_Quest_Format 5. Duckstation Cheat Code Guide | PDF | Computer Engineering | Arithmetic - Scribd, <https://www.scribd.com/document/649557055/chtddb> 6. Attempt at Python tooling for the Dragon Quest 4 PSX Translation - GitHub, <https://github.com/mwilkens/dq4psxtrans> 7. Dragon Quest V - Dragon Quest Wiki - Fandom, https://dragonquest.fandom.com/wiki/Dragon_Quest_V 8. damnn....i want this level of expertise : r/ProgrammerHumor - Reddit, https://www.reddit.com/r/ProgrammerHumor/comments/xf4u1t/damnni_want_this_level_of_expertise/ 9. Dragonlord - Dragon Quest Wiki, <https://dragon-quest.org/wiki/Dragonlord> 10. Dragon Quest VII: Sound Engine Analysis - GitHub Gist, <https://gist.github.com/loveemu/582ff7683a797b7048b6> 11. 3DS DQ7 Save Editor English Translation - GitHub, <https://github.com/Gravitinox/DQ7-Save-Editor-English-Translation>